

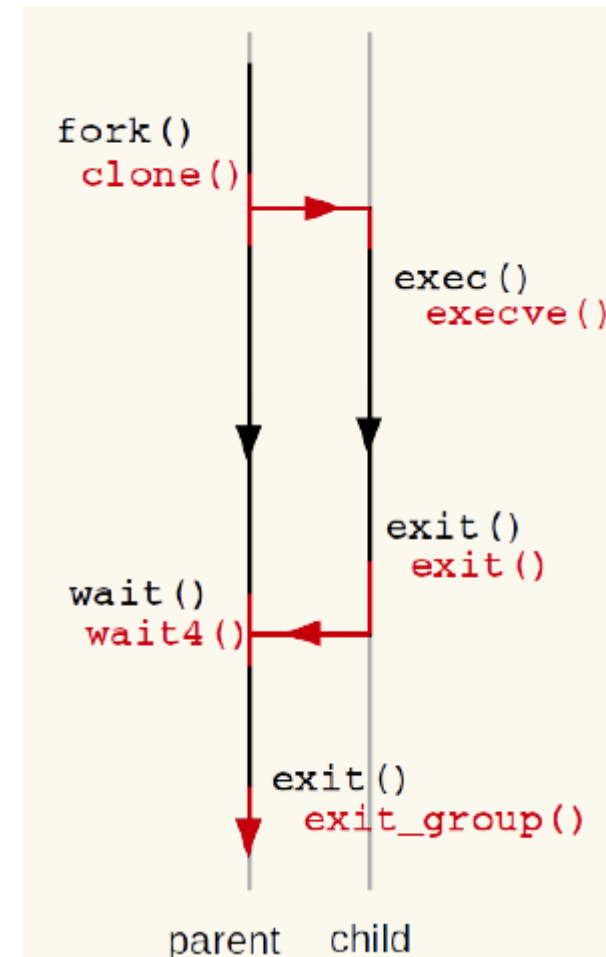
Process Management

MAST

Lakshmi Prasanna E

Process Management

- Refers to a **program currently executing in the system**
 - ✓ CPU registers
 - ✓ Location and state of memory segments (text, data, stack, etc.)
 - ✓ Kernel resources (open files, pending signals, etc.)
 - ✓ **Threads**
- Managed on a per-program
 - Virtualization of the processor and the memory
- ▶ Example with strace (-f)



Process Management

- `#include <sys/types.h> // For pid_t data type`
`#include <unistd.h> // For fork(), getpid(), getppid()`
`#include <sys/wait.h> // For wait() function`
`#include <stdio.h> // For standard I/O`
`#include <stdlib.h> // For exit()`

Always use `pid_t` instead of `int` for process IDs. This is because:

`pid_t` is the designated type for process IDs in UNIX systems

It ensures portability across different systems

Process Management

➤ Basic Process Creation

```
int main() {
    pid_t pid = fork();

    // Error handling - always check fork's return value
    if (pid == -1) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }

    // Child process
    if (pid == 0) {
        printf("Child Process:\n");
        printf("PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }

    // Parent process
    else {
        printf("Parent Process:\n");
        printf("PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        // Wait for child to complete
        wait(NULL);
    }

    return 0;
}
```

Process Management

- List of processes implemented as a linked list of task struct
- Total size (Linux 4.8): 6976 bytes
 - Full structure definition in linux/include/linux/sched.h

```
1 struct task_struct {
2     volatile long state;
3     void *stack;
4     /* ... */
5     int prio;
6     /* ... */
7     cpumask_t cpus_allowed;
8     /* ... */
9     struct list_head tasks;
10    /* ... */
11    struct mm_struct *mm;
12    /* ... */
13    pid_t pid;
14    /* ... */
15    struct task_struct *parent;
16    struct list_head children;
17    struct list_head sibling;
18    /* ... */
19 }
```

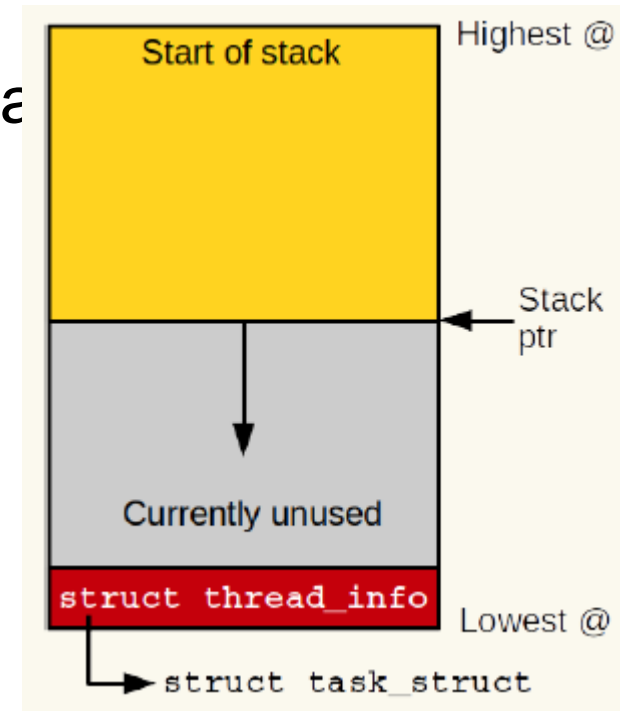
Process Management

- Prior to 2.6: task struct allocated at the end of the kernel stack of each process
 - ✓ Allows to retrieve it without storing its location in a register.

- Now dynamically allocated (heap) through the slab allocator
 - ✓ A struct thread info living at the bottom of the stack

```
struct thread_info {  
    struct task_struct *task;  
    __u32 flags;  
    __u32 status;  
    __u32 cpu;  
};
```

- Moved off the stack in 4.9 [2] because of potential exploit [1] when overflowing the kernel stack



Process Management

- **Process Identifier (PID):** pid_t (int)
 - Max: 32768, can be increased to 4 millions
 - Wraps around when maximum reached
- **Quick access to** task_struct of the task currently running on a core:
- [linux/arch/x86/include/asm/current.h at master · torvalds/linux · GitHub](#)

```
DECLARE_PER_CPU_CACHE_HOT(struct task_struct *, current_task);
/* const-qualified alias provided by the linker. */
DECLARE_PER_CPU_CACHE_HOT(struct task_struct * const __percpu_seg_override,
                           const_current_task);

static __always_inline struct task_struct *get_current(void)
{
    if (IS_ENABLED(CONFIG_USE_X86_SEG_SUPPORT))
        return this_cpu_read_const(const_current_task);

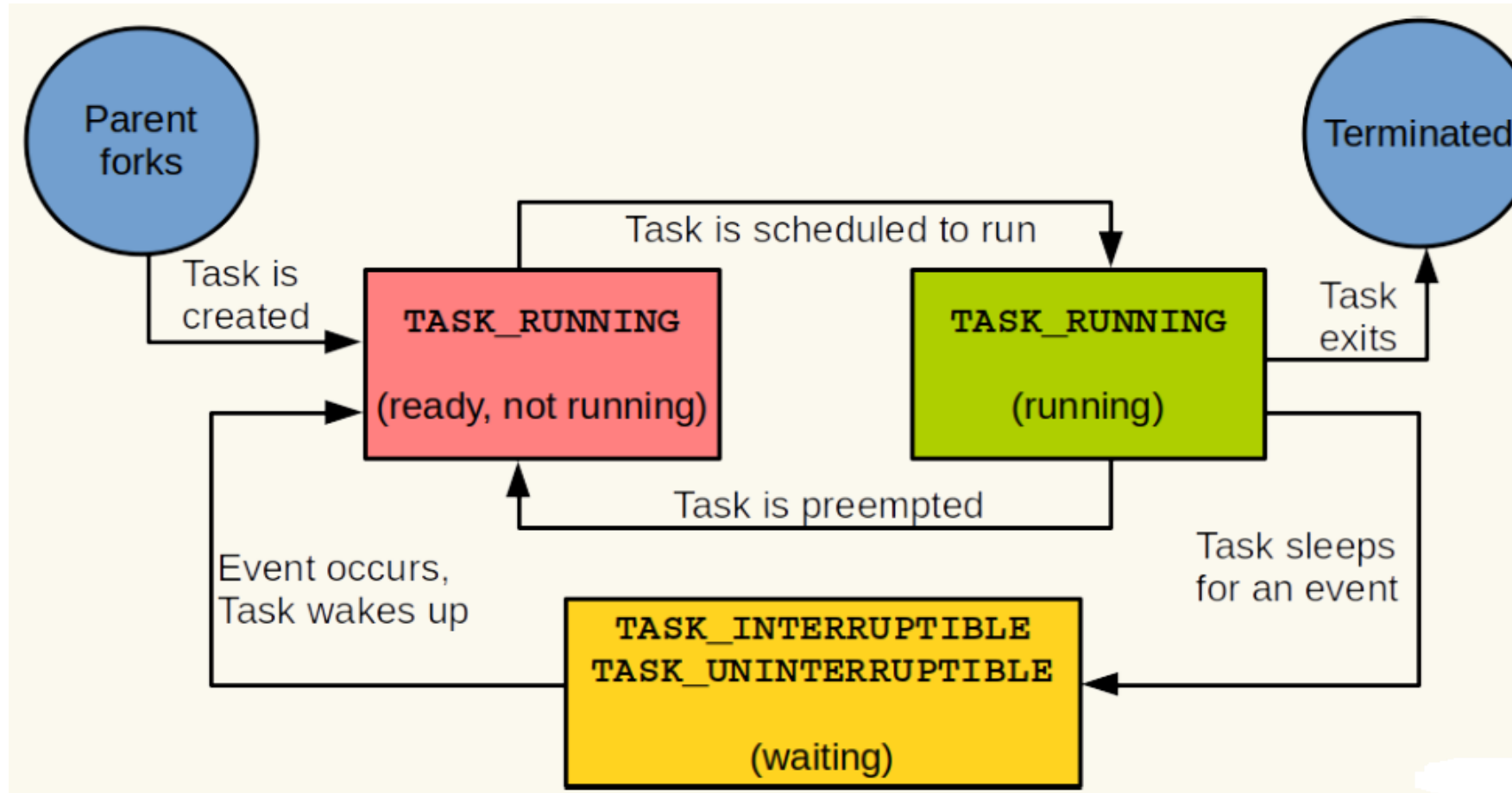
    return this_cpu_read_stable(current_task);
}

#define current get_current()
```

Process Management

- **state** field of the task struct
 - **TASK RUNNING:**
 - Process is runnable (running or in a CPU run queue)
 - In user or kernel space
 - **TASK INTERRUPTIBLE**
 - Process is sleeping waiting for some condition
 - Switched to TASK RUNNING on condition true or signal received
 - **TASK UNINTERRUPTIBLE :**
 - Same as TASK INTERRUPTIBLE but does not wakeup on signal
 - **TASK TRACED:**
 - Traced by another process
 - **TASK STOPPED:**
 - Not running nor waiting, result of the reception of some signals to pause the process
 - **TASK ZOMBIE:**
 - Parent process is closed but child process is not closed
 - Ps aux | grep 'z'

Process Management



Process Management

- The kernel can execute in **process** vs **interrupt** context
 - current is meaningful only when the kernel executes in process context
 - I.e. following a system call or an exception
- Process hierarchy
 - Root: init, PID 1
 - Launched by the kernel as the last step of the boot process
- fork-based process creation:
 - Each process has a parent: parent pointer in the task struct
 - Processes may have children: children field (list head)
 - Processes may have siblings: siblings' field
 - List of all tasks: tasks field
 - - Easy manipulation through next task(t) and for each process(t)

Process Management

- Linux does not implements creating a tasks from nothing (spawn)
- **fork() & exec()**
 - fork() creates a child, copy of the parent process
 - Only PID, PPID and some resources/stats differ
 - exec() loads into a process address space a new executable
- On fork(), Linux duplicates the parent page tables and creates a new process descriptor
 - It's fast, as **the address space is not copied**
 - Page table access bits: read-only
 - **Copy-On-Write** (COW): m
- fork() is implemented by the **clone()** system call
- sys clone() calls do fork(), which calls **copy process()** and starts the new task memory pages are copied only when they are referenced for write operations

Process Management

copy process():

- Calls dup task struct()
- Duplicates kernel stack, task struct and thread info
- Checks that we do not overflow the processes number limit
- Small number of values are modified in the task struct
- Calls sched fork() to set the child state set to TASK NEW
- Copies parent info: files, signal handlers, etc.
- Gets a new PID through alloc pid()
- Returns a pointer to the created child task struct
- Finally, do fork() calls wake up new task()
 - State becomes TASK RUNNING
- vfork(): alternative without copy of the address space

Process Management

- Fork can return three different values:
 - -1: Fork failed (error condition)
 - 0: We're in the child process
 - Positive number: We're in the parent process (number is the child's PID)

Key differences of process and threads

Feature	Process	Thread (User-Level)	Kernel Thread
Memory Space	Separate	Shared	Shared
Creation Overhead	High	Low	Medium
Scheduling	Kernel	User or Kernel	Kernel
Communication	IPC	Shared memory	Shared memory
Crash Impact	Isolated	Affects entire process	Affects entire process
Linux View	PID	TID (Thread ID)	TID (Thread ID)

Process Management

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 < 0) {
        perror("First fork failed");
        exit(1);
    }
    if (pid1 == 0) {
        // First child process
        printf("First Child Process:\n");
        printf("PID: %d, Parent PID: %d\n", getpid(), getppid());
        // Create another child (grandchild of original process)
        pid2 = fork();
        if (pid2 < 0) {
            perror("Second fork failed");
            exit(1);
        }
    }
}
```

Process Management

```
if (pid2 == 0) {  
    // Grandchild process  
    printf("Grandchild Process:\n");  
    printf("PID: %d, Parent PID: %d\n", getpid(), getppid());  
} else {  
    wait(NULL); // Wait for grandchild  
}  
} else {  
    wait(NULL); // Wait for first child  
    printf("Parent Process:\n");  
    printf("PID: %d\n", getpid());  
}  
  
return 0;  
}
```

Process Management

- Termination on invoking the `exit()` system call
 - Can be implicitly inserted by the compiler on return from `main`
 - `sys exit()` calls `do exit()`
- `do exit()` (`kernel/exit.c`):
 - Calls `exit signals()` which set the `PF EXITING` flag in the task struct
 - Set the exit code in the exit code field of the task struct
 - To be retrieved by the parent
 - Calls `exit mm()` to release the mm struct for the task
 - If it is not shared with any other process, it is destroyed
 - Calls `exit sem()`: process dequeued from potential semaphores n queues
 - Calls `exit fs()` and `exit files()` to update accounting
 - information
 - Potential data structures that are not used anymore are freed

Process Management

- do exit() :
 - Calls exit_notify()
 - Sends signals to parent
 - Reparent potential children
 - Set the exit state of the task struct to EXIT_ZOMBIE
- Calls do_task_dead()
 - Sets the state to TASK_DEAD
 - Calls schedule() and never returns
- At that point, what is left is the task struct, thread info and kernel stack
 - To provide information to the parent
 - Parent notifies the kernel when everything can be freed
- Separated from the process of exiting because of the need to
- pass exit information to the parent
 - task struct must survive a little bit before being deallocated.
Until the parent grab the exit information through wait4()
- Cleanup implemented in release_task() called from the wait4() implementation
- Remove the task from the task list and Release and free remaining resources

Process Management

- **A parent exits before its child**
 - Child must be reparented
 - To another process in the current thread group ...
 - ... or init if that fails exit notify() calls forget original parent(), that calls find new reaper()
- Returns the task struct of another task in the thread group if it exists, otherwise the one from init
- Then, all the children of the currently dying task are reparented to the reaper